



CI/CD for AI Systems

Tự động hoá vòng đời model: từ thí nghiệm đến production

AICB-P2T2 · Ngày 21 · Chương 5: Vận Hành

HÃY SUY NGHĨ...

***"Code thay đổi mỗi ngày — model cũng vậy.
CI/CD cho AI khác gì CI/CD cho software thông thường?"***

Case Study:

Một team deploy model mới mỗi tuần bằng tay.

- 3 lần model bị regression nhưng không ai biết đến khi user phản hồi.
- Sau khi có CI/CD pipeline: ZERO regression lọt production trong 6 tháng.

Giữ câu hỏi này trong đầu suốt buổi học hôm nay

Nội Dung Bài Học

01

MLflow Experiment Tracking & Model Registry

30 min

02

DVC Data Version Control & Pipelines

30 min

03

GitHub Actions CI Pipeline cho AI

30 min

04

CD Model Deployment Strategies

20 min

05

Testing Pyramid cho AI Systems

15 min

06

MLflow Model Serving & A/B Testing

15 min

07

Live Demo: Full CI/CD Pipeline

20 min

08

Key Takeaways & Preview Ngày 22

10 min

Mục Tiêu Bài Học

Sau buổi học này, bạn sẽ có thể:

- 01 Setup MLflow tracking server và log experiments một cách có hệ thống, so sánh runs qua UI.
- 02 Implement DVC cho data versioning, tạo reproducible ML pipelines với dvc.yaml.
- 03 Build GitHub Actions CI/CD pipeline tự động test, train, eval và deploy AI models.
- 04 Áp dụng các deployment strategies (canary, blue/green, shadow) để giảm rủi ro khi release model mới.

Deliverable Cuối Ngày

MLflow tracking server + DVC-versioned dataset + GitHub Actions pipeline chạy auto test/deploy

MLflow UI

≥ 3 tracked experiments với params, metrics, và artifacts được log đầy đủ

DVC Pipeline

3 stages (prepare → train → evaluate) chạy dvc repro thành công

GitHub Actions

Workflow pass: test → train → eval gate → deploy trên repo thực

Model Registry

Model được register trong MLflow Registry và promote lên Staging

01

MLflow: Experiment Tracking & Model Registry

Theo dõi, so sánh, và quản lý vòng đời model một cách hệ thống

MLflow: 4 Thành Phần Cốt Lõi

MLflow là open-source platform để quản lý toàn bộ vòng đời ML — từ thí nghiệm đến production.

Tracking

Log parameters, metrics, artifacts và source code.
So sánh runs qua web UI hoặc API.

Projects

Đóng gói code để reproducibility.
Chạy lại bất kỳ run nào trên bất kỳ platform nào.

Models

Chuẩn hoá format packaging (mlflow.sklearn, .pytorch, .pyfunc).
Deploy lên nhiều nền tảng khác nhau.

Registry

Quản lý lifecycle: None → Staging → Production → Archived.
Workflow review & approve trước khi promote.

MLflow Tracking: Log Experiments

```
import mlflow
from mlflow.models import infer_signature

mlflow.set_experiment("sentiment-v2")

with mlflow.start_run(run_name="lr_0001_ep10"):
    # Log hyperparameters
    mlflow.log_param("lr", 0.001)
    mlflow.log_param("epochs", 10)
    mlflow.log_param("batch_size", 32)

    # Training loop
    for epoch in range(10):
        loss = train_one_epoch(model, loader)
        acc = evaluate(model, val_loader)

        mlflow.log_metric("train_loss", loss, step=epoch)
        mlflow.log_metric("val_accuracy", acc, step=epoch)

    # Save artifacts & model
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.sklearn.log_model(
        model, "model",
        signature=infer_signature(X_val, y_pred)
    )
```

Params

Hyperparameters, data version, model config — bất biến trong một run

Metrics

Loss, accuracy — log tại mỗi step/epoch để vẽ đồ thị so sánh

Artifacts

Plots, confusion matrix, config files lưu cùng run để trace back

Signature

Input/output schema tự động suy diễn — cần thiết cho serving

MLflow: So Sánh Runs & LLM Autolog

So Sánh Runs trong MLflow UI

- Filter & sort runs theo bất kỳ metric nào
- Parallel Coordinates Chart — thấy ngay lr nào cho loss thấp nhất
- Scatter Plot — compare accuracy vs latency
- Diff params giữa 2 runs để debug regression
- Download artifacts, view confusion matrix trực tiếp trên UI

LLM Autolog (MLflow 2.8+)

```
# OpenAI autolog
mlflow.openai.autolog()

# LangChain autolog
mlflow.langchain.autolog()

# Tự động log:
# - Prompt templates
# - Input / output content
# - Token usage (prompt, completion)
# - Latency per call
# - Model name & version
# - Retrieval context (RAG)
```

→ Không cần sửa code, chỉ cần gọi `autolog()` trước khi run

→ Hỗ trợ: *OpenAI, LangChain, LlamaIndex, Anthropic Claude*

MLflow Model Registry: Lifecycle Management



Mỗi version

Có stage riêng. Nhiều version cùng tồn tại — team có thể A/B test champion vs challenger.

Annotations

Ghi lý do promote/reject, link đến eval report, người approve — full audit trail.

Alias

champion = Production version, challenger = A/B candidate. Load bằng tên alias.

Webhook

Trigger CI/CD khi model chuyển stage — tự động deploy staging khi Staging được approve.

MLflow Registry: API & Best Practices

```
import mlflow
from mlflow import MlflowClient

client = MlflowClient()

# Register model từ run
result = mlflow.register_model(
    model_uri=f"runs:{run_id}/model",
    name="sentiment_classifier"
)

# Promote to Staging
client.transition_model_version_stage(
    name="sentiment_classifier",
    version=result.version,
    stage="Staging"
)

# Load model bằng alias "champion"
model = mlflow.sklearn.load_model(
    model_uri="models:/sentiment_classifier@champion"
)
```

Naming Convention

Dùng tên model rõ ràng: {task}_{arch}_{version}
VD: sentiment_bert_base, fraud_lgbm_v2

Tag Strategy

Tag với: data_version, git_commit, eval_accuracy
→ Trace back ngay khi có incident

Approval Workflow

Require ≥1 reviewer approve trước khi promote
Staging → Production
Dùng webhook để notify Slack

Rollback Plan

Luôn giữ previous Production version ở Archived
→ 1 API call để rollback nếu cần

02

DVC: Data Version Control

Git cho data — versioning, pipelines, và reproducibility

Vấn Đề: Data Không Có Version Control

? Không tái tạo được kết quả

"Hôm qua model đạt 92% nhưng hôm nay chỉ còn 88%" — không biết data đã thay đổi hay code?

! Merge conflict với data lớn

Git không track được file GB. Team dùng shared folder → overwrite nhau, không có history.

× Experiment không gắn với data

MLflow log metrics nhưng không biết train trên data version nào → A/B comparison vô nghĩa.

\$ Storage lãng phí

Mỗi người copy data riêng → duplicates tốn hàng chục GB. Không có deduplication.

DVC: Git cho Data

```
# 1. Khởi tạo DVC trong repo
git init && dvc init

# 2. Add data file (tạo .dvc pointer)
dvc add data/training_set.parquet
# → Tạo data/training_set.parquet.dvc
# → Thêm data/ vào .gitignore tự động

# 3. Commit .dvc file vào git
git add data/training_set.parquet.dvc .gitignore
git commit -m "track training dataset v1"

# 4. Push data lên remote storage
dvc push

# 5. Team member pull data
git clone <repo>
dvc pull # download đúng version data
```

Pointer File (.dvc)

File nhỏ lưu hash của data. Git track .dvc file → checkout code = checkout data đúng version.

Content-Addressable

Data lưu theo hash nội dung → không tốn dung lượng cho duplicates dù có 100 versions.

Remote Supports

Amazon S3, Google GCS, Azure Blob, SSH server, HDFS — cùng API dvc push/pull.

Offline-first

Làm việc local không cần mạng. Sync remote khi cần — giống git push/pull.

DVC Remote Storage: Cấu Hình

```
# Setup S3 remote (khuyến nghị cho production)
dvc remote add -d myremote s3://my-bucket/dvc-store
dvc remote modify myremote region us-east-1

# Google Cloud Storage
dvc remote add gcs_remote gs://my-gcs-bucket/data

# Azure Blob Storage
dvc remote add azure_remote azure://mycontainer/data

# SSH server (on-premise)
dvc remote add ssh_remote ssh://user@server:/path/data

# Xem danh sách remotes
dvc remote list

# Push / pull
dvc push                # upload tất cả data
dvc pull data/train.parquet # pull một file cụ thể
```

Best Practices cho Remote Storage

- Dùng S3/GCS cho team production — có versioning và ACL
- Cấu hình IAM role cho CI/CD, không dùng access key cứng
- Bật S3 server-side encryption cho data nhạy cảm
- Dùng `dvc remote modify --local` để lưu credentials local (không commit)
- Set lifecycle policy trên S3: move cold data sang Glacier sau 90 ngày
- Cache locally với DVCCACHE — tránh re-download khi checkout branches
- Tách remote: dev (fast) và archive (cheap cold storage)

DVC Pipeline: Reproducible Workflows

```
# dvc.yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - data/raw/
      - src/prepare.py
    outs:
      - data/processed/

  train:
    cmd: python src/train.py --lr ${params.lr}
    deps:
      - data/processed/
      - src/train.py
      - params.yaml
    outs:
      - models/model.pkl
    metrics:
      - metrics.json:
        cache: false

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - models/model.pkl
      - data/test/

    metrics:
      - eval_metrics.json:
```

`dvc repro`

Chạy lại stages bị stale (deps thay đổi). Smart caching — bỏ qua stages chưa thay đổi.

`dvc dag`

Visualize pipeline DAG trong terminal. Thấy ngay dependency graph.

`dvc metrics show`

So sánh metrics.json giữa các lần chạy. Hiện diff rõ ràng.

`dvc params diff HEAD~1`

So sánh params giữa current và previous commit.

DVC Experiments: So Sánh Hyperparameters

```
# Chạy experiment với param khác nhau
dvc exp run --set-param lr=0.001
dvc exp run --set-param lr=0.01
dvc exp run --set-param lr=0.1 --set-param epochs=20

# Chạy nhiều experiments song song
dvc exp run --set-param lr=0.001,0.01,0.1 --jobs 3

# Xem kết quả so sánh
dvc exp show
#
```

#	Experiment	lr	epochs	accuracy
#	workspace	0.01	10	0.924
#	exp-abc123	0.001	10	0.918
#	exp-def456	0.1	10	0.891
#	exp-ghi789	0.1	20	0.912

```
# Apply kết quả tốt nhất vào workspace
dvc exp apply exp-abc123

# Persist experiment thành branch
dvc exp branch exp-abc123 best-lr-001
```

Git + DVC Workflow

- git commit → code version
- dvc push → data version
- .dvc file liên kết 2 layer lại
- Bất kỳ ai git clone + dvc pull
→ reproduce y hệt kết quả

DVC vs MLflow:

DVC: pipeline + data versioning

MLflow: metrics + model registry

→ Dùng cả hai, không chọn một

03

GitHub Actions: CI Pipeline cho AI

Tự động hoá test, train và eval mỗi khi code thay đổi

CI/CD cho AI: Khác Gì Software Thông Thường?

Khía cạnh	CI/CD Software truyền thống	CI/CD cho AI
Artifact	Binary / Docker image	Model weights + metadata
Test	Unit test, integration test	+ Model eval, data validation
Versioning	Git cho code	Git + DVC cho code + data
Deploy	Deterministic — code giống nhau	Non-deterministic — cần eval gate
Rollback trigger	Error rate tăng	+ Accuracy drop, bias metrics
Pipeline input	Code changes only	Code hoặc data changes
Thời gian build	Vài phút	Vài phút → vài giờ (training)

CI Pipeline Architecture cho AI Projects



Trigger

on: push (main), pull_request (main) → mọi PR đều qua pipeline trước khi merge

Path Filter

Training job chỉ chạy khi data/ hoặc src/ thay đổi — tránh retrain khi sửa docs

Fail Fast

Data Validation fail → dừng pipeline ngay, không chạy training tốn GPU

Eval Gate

Số sánh new model vs production baseline — block deploy nếu accuracy drop >2%

GitHub Actions: Cấu Trúc Workflow YAML

```
name: AI CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
  AWS_ACCESS_KEY_ID:   ${ secrets.AWS_ACCESS_KEY_ID }

jobs:
  data-validation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: pip-${ hashFiles('requirements.txt') }
      - run: pip install -r requirements.txt
      - run: dvc pull data/
      - run: great_expectations checkpoint run data_quality
        # Fail fast nếu data drift detected
```

Secrets Management

MLFLOW_TRACKING_URI, AWS keys →
GitHub Secrets.
KHÔNG hardcode trong workflow file.

Dependency Cache

actions/cache với key = hash(requirements.txt)
→ Giảm 60-80% thời gian setup.

Job Dependencies

needs: [data-validation] → training chỉ chạy khi
validation pass. Sequential control flow.

OIDC Federation

Production: dùng OIDC thay vì long-lived AWS
keys. Hết hạn sau mỗi run.

CI Job: Data Validation với Great Expectations

```
# .github/workflows/ai-cicd.yml (tiếp theo)
train:
  needs: [data-validation]
  runs-on: ubuntu-latest
  if: |
    github.event_name == 'push' &&
    contains(toJson(github.event.commits.*.modified),
      'data/') ||
    contains(toJson(github.event.commits.*.modified),
      'src/')

  steps:
    - uses: actions/checkout@v4
    - run: dvc pull data/
    - run: dvc repro train
    - run: |
        mlflow run . --entry-point train \
        -P lr=0.001 -P epochs=10

# great_expectations/checkpoints/data_quality.yml
name: data_quality
class_name: Checkpoint
validations:
  - batch_request:
      datasource_name: training_data
      expectation_suite_name: training.warning
    action_list:
      - name: store_validation_result
        action: { class_name: StoreValidationResultAction }
      - name: send_slack_alert
        action: { class_name: SlackNotificationAction }
```

Các kiểm tra Data Quality

- Schema validation: đúng columns, đúng dtypes
- Null rate: mỗi column $\leq 5\%$ null
- Value range: tuổi 0–120, price > 0
- Distribution drift: KL-divergence vs baseline
- Duplicate rows $< 0.1\%$
- Label balance: minority class $\geq 10\%$
- Volume check: $\geq 10,000$ rows mỗi batch
- Freshness: data không cũ hơn 7 ngày

CI Job: Eval Gate — Safety Net Quan Trọng Nhất

```
# compare_models.py
import mlflow, sys, json

def eval_gate(new_model_uri, prod_model_uri,
              threshold=0.02):
    client = mlflow.MlflowClient()

    # Load cả hai models
    new_model = mlflow.sklearn.load_model(new_model_uri)
    prod_model = mlflow.sklearn.load_model(prod_model_uri)

    # Evaluate trên held-out test set
    new_acc = evaluate(new_model, X_test, y_test)
    prod_acc = evaluate(prod_model, X_test, y_test)

    delta = new_acc - prod_acc
    print(f"New: {new_acc:.4f} | Prod: {prod_acc:.4f} | Δ={delta:+.4f}")

    if delta < -threshold:
        print(f"FAIL: accuracy drop {delta:.4f} > threshold {-threshold}")
        sys.exit(1) # GitHub Actions job fails → blocks deploy

    print("PASS: new model >= baseline - threshold")

# GitHub Actions step:
# - run: python compare_models.py
# - new_model: /sentiment/${cat new_version.txt}
# - prod_model: /sentiment@champion
```

Eval Gate Best Practices

- Dùng FIXED held-out test set — không shuffle mỗi lần eval
- Track nhiều metrics: accuracy, F1, AUC, latency P95
- Threshold khác nhau theo metric: accuracy $\pm 2\%$, latency $\pm 10\%$
- Log eval results vào MLflow để có history
- Comment kết quả vào PR để reviewer thấy
- Nếu pass borderline: yêu cầu manual review thay vì auto-deploy

CI Best Practices cho AI Repositories

Secrets & Credentials

GitHub Secrets cho MLFLOW_TRACKING_URI, AWS keys.
Dùng OIDC federation cho production — không có long-lived keys.

Caching Strategy

actions/cache với pip (key=hash requirements), DVC cache trên S3.
Self-hosted runner nếu cần GPU — tiết kiệm 10x cost.

Matrix Builds

Test trên Python 3.10, 3.11, 3.12.
Matrix strategy cho AI: test across model sizes, CUDA versions.

Conditional Execution

Path filters: train job chỉ chạy khi src/ hoặc data/ thay đổi.
Doc-only PR bỏ qua training → tiết kiệm tiền GPU.

Parallelism

Chạy unit test, lint, data validation song song (jobs độc lập).
Fail fast: cancel running jobs khi một job fail.

Notifications

Slack webhook khi pipeline fail, khi eval gate block deploy.
Post kết quả eval vào PR comment tự động.

04

CD: Model Deployment Strategies

Giảm rủi ro khi deploy model mới vào production

4 Deployment Strategies cho AI Models

Canary

Cách hoạt động: Route 5% traffic → model mới, tăng dần khi healthy

✓ Phát hiện lỗi sớm, kiểm soát risk từng bước

✗ Cần monitoring tốt, traffic split logic phức tạp

Blue/Green

Cách hoạt động: Deploy v2 song song v1, switch load balancer khi ready

✓ Zero downtime, instant rollback qua load balancer

✗ Tốn gấp đôi resource khi cả hai env chạy song song

Shadow

Cách hoạt động: Model mới xử lý traffic nhưng không trả response

✓ Test với real traffic không ảnh hưởng user

✗ Không test user reaction, tốn compute gấp đôi

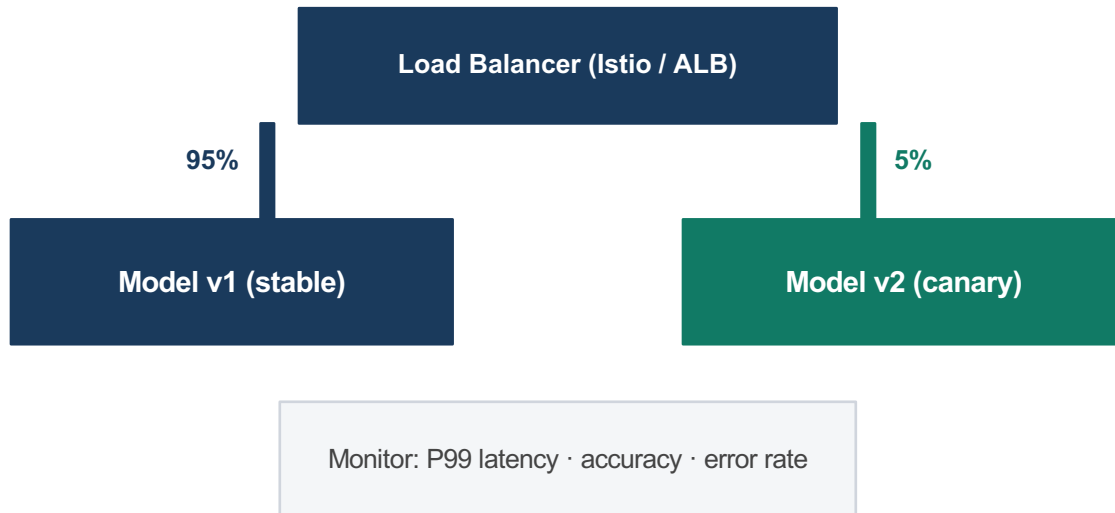
Rolling

Cách hoạt động: Thay từng pod một trong Kubernetes deployment

✓ Đơn giản, default K8s, không cần extra infra

✗ Slow rollback, mixed versions trong quá trình deploy

Canary Deployment: Deep-Dive



Rollout Steps:



Rollback trigger: P99 latency vượt threshold HOẶC accuracy drop >2% → auto rollback về v1

Blue/Green & Shadow Mode: Chi Tiết

Blue/Green Deployment

- Deploy v2 hoàn toàn độc lập với v1 đang chạy
- Chạy smoke test & integration test trên v2
- Switch load balancer → toàn bộ traffic sang v2
- Keep v1 sống thêm 30 phút → instant rollback nếu cần
- Sau 30 phút stable → shutdown v1, giải phóng resource

Khi nào dùng: deploy lớn, cần zero downtime tuyệt đối

Trade-off: tốn gấp đôi infrastructure cost trong khi switch

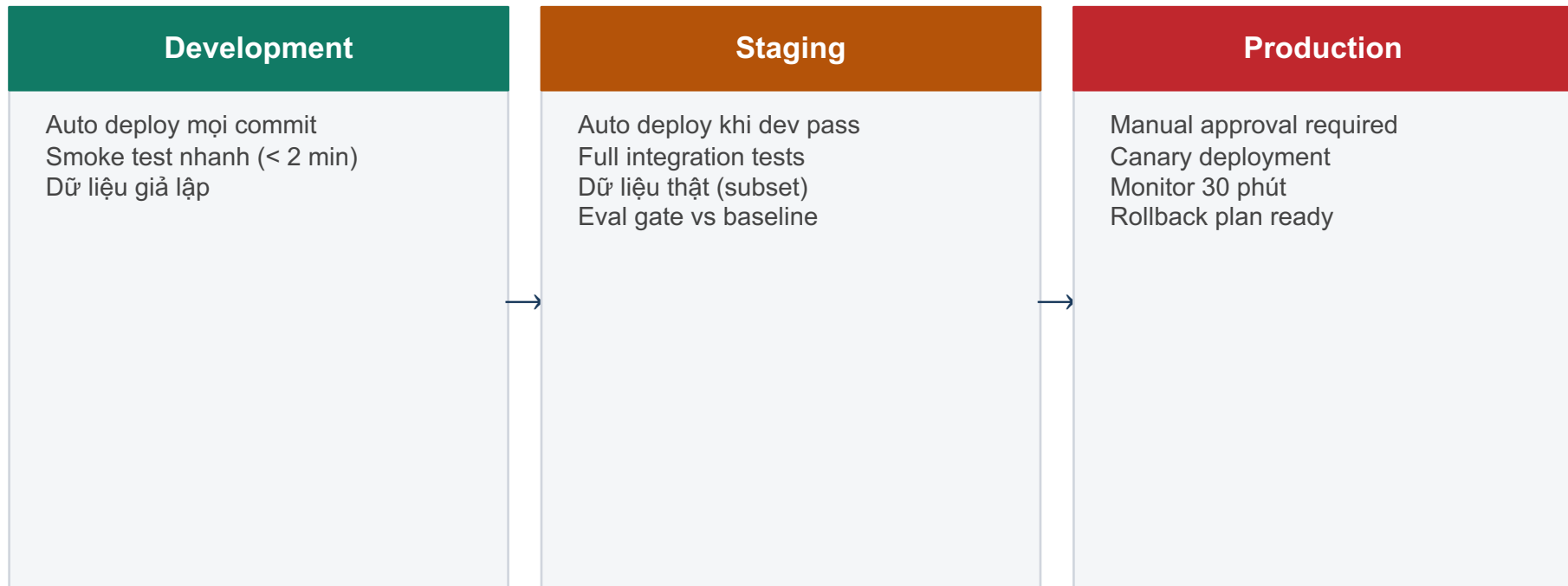
Shadow Mode (Dark Launch)

- Request được forward tới CẢ HAI models
- Response của v1 trả về user như bình thường
- Response của v2 (shadow) chỉ được log, không trả user
- So sánh outputs: latency, predictions, errors
- Hoàn toàn zero risk với user experience

Khi nào dùng: model thay đổi lớn, muốn validate trước

Trade-off: tốn compute gấp đôi, không test user reaction

Multi-Environment CD Pipeline



GitOps cho AI với ArgoCD / Flux:

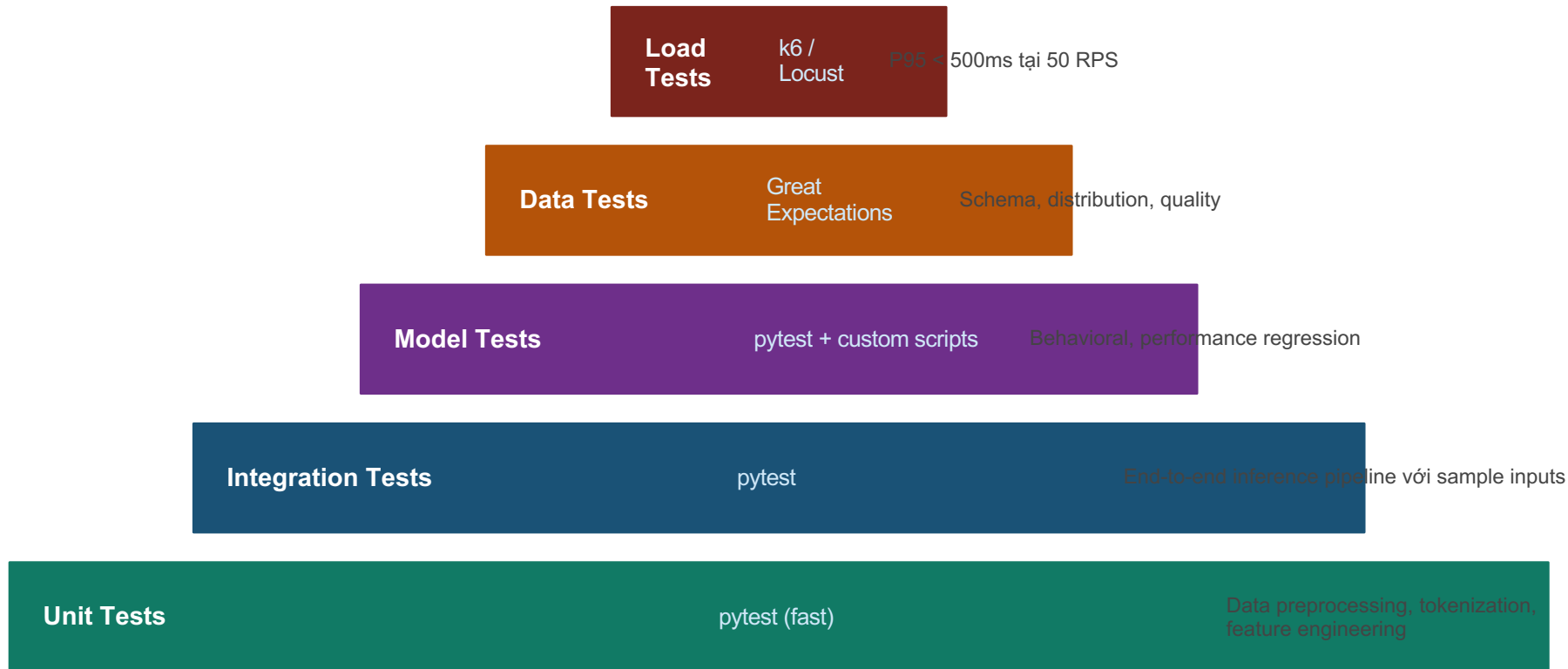
Declarative deployment — mọi thay đổi infra đều qua git commit → auto-sync từ repo → rollback bằng git revert

05

Testing Pyramid cho AI Systems

Từ unit test đến load test — đảm bảo chất lượng ở mọi tầng

AI Testing Pyramid: Tổng Quan



Unit Tests & Integration Tests

```
# Unit tests: test từng function độc lập
import pytest
from src.preprocessing import clean_text, tokenize

def test_clean_text_removes_html():
    assert clean_text("<b>hello</b>") == "hello"

def test_tokenize_max_length():
    tokens = tokenize("word " * 600, max_len=512)
    assert len(tokens) <= 512

def test_feature_engineering_no_leakage():
    """Test rằng feature engineer không dùng future data"""
    df = make_test_df(n=100)
    features = engineer_features(df, target_col="label")
    assert "label" not in features.columns

# Integration tests: test toàn bộ pipeline
def test_inference_pipeline_e2e():
    """Full pipeline: raw input → prediction"""
    payload = {"text": "This product is great!"}
    response = client.post("/predict", json=payload)
    assert response.status_code == 200
    assert "label" in response.json()
    assert "confidence" in response.json()
    assert 0.0 <= response.json()["confidence"] <= 1.0
```

Checklist cho Unit Tests

- Test tất cả preprocessing steps riêng lẻ
- Test edge cases: empty input, null, max length
- Test tokenization với special characters, tiếng Việt
- Test feature engineering: không data leakage
- Mock external calls (API, DB) trong unit tests
- Coverage $\geq 80\%$ cho src/ module
- Chạy dưới 30 giây cho toàn bộ unit test suite
- Integration test: dùng fixture nhỏ (100 samples)

Model Tests · Data Tests · Load Tests

Model Tests — Behavioral & Regression

- Behavioral: model PHẢI từ chối nội dung harmful
- Invariance: xoay ảnh 90° → prediction không đổi
- Directional: thêm 'not' vào câu → sentiment đổi chiều
- Regression: accuracy trên golden test set $\geq v_{\text{prev}} - 0.5\%$
- Fairness: accuracy gap giữa subgroups $< 3\%$

Data Tests — Great Expectations

- Schema: đúng columns, đúng dtypes, không extra columns
- Completeness: null rate mỗi column $\leq$ threshold
- Distribution: KS test so với baseline distribution
- Volume: số rows trong expected range [min, max]
- Chạy trên mỗi new data version, log kết quả vào MLflow

Load Tests — k6 / Locust

- Baseline: 50 RPS, P95 $< 500\text{ms}$, error rate $< 0.1\%$
- Stress test: tăng dần đến 500 RPS tìm breaking point
- Soak test: 50 RPS trong 1 giờ — check memory leak
- Spike test: 0 → 500 RPS đột ngột — check auto-scaling
- Fail pipeline nếu bất kỳ SLA nào bị vi phạm

06

MLflow Model Serving & A/B Testing

Từ model registry đến production endpoint, A/B test và gradual rollout

MLflow Model Serving: Các Tùy Chọn

```
# Local serving - phát triển và testing
mlflow models serve \
  -m "models:/sentiment_classifier/Production" \
  -p 5000 --no-conda

# Test endpoint
curl -X POST http://localhost:5000/invocations \
  -H "Content-Type: application/json" \
  -d '{"inputs": [{"text": "Great product!"}]}'

# Build Docker image
mlflow models build-docker \
  -m "models:/sentiment_classifier/Production" \
  -n sentiment-serving:v1.0

# Deploy lên Kubernetes với KServe
kubectl apply -f - <<EOF
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sentiment-classifier
spec:
  predictor:
    model:
      storageUri: "s3://models/sentiment/v1"
      modelFormat: { name: mlflow }
    resources:
      requests: { cpu: "1", memory: "2Gi" }
```

Local

mlflow models serve
Dành cho dev/test. Khởi động nhanh.

Docker

mlflow models build-docker
Portable, dùng cho staging deploy.

Cloud

AWS SageMaker, Azure ML, Databricks
Managed infrastructure, auto-scaling.

Kubernetes

Seldon Core, KServe
Production-grade, custom scaling policy.

A/B Testing cho AI Models

```
import hashlib, mlflow

def route_request(user_id: str, pct_b: float = 0.1):
    """Route user to variant A or B deterministically."""
    h = int(hashlib.md5(user_id.encode()).hexdigest(), 16)
    return "B" if (h % 100) < (pct_b * 100) else "A"

def predict(user_id: str, text: str):
    variant = route_request(user_id)
    model = model_b if variant == "B" else model_a

    with mlflow.start_run(run_id=EXPERIMENT_RUN_ID):
        prediction = model.predict([text])
        # Log outcome for statistical analysis
        mlflow.log_metric(f"click_{variant}", 1)
        mlflow.log_metric(f"latency_{variant}", elapsed_ms)

    return prediction

# Statistical significance check
from scipy import stats
chi2, p_value = stats.chi2_contingency(contingency_table)
if p_value < 0.05:
    print(f"Statistically significant at 95% confidence")
    print(f"Winner: {'B' if ctr_b > ctr_a else 'A'}")
```

Traffic Routing

Hash user_id → deterministic assignment.
Same user luôn thấy cùng variant (consistency).

Sample Size

Minimum 1,000 samples/variant trước khi kết luận.
Dùng power analysis để tính trước.

Significance Level

p-value < 0.05 (95% confidence) để declare winner.
Corridor cho multiple comparisons: Bonferroni.

Metric Selection

Primary: business KPI (CTR, revenue).
Guardrail: latency P95, error rate không được tăng.

07

Live Demo: Full CI/CD Pipeline

git push → production trong 8 phút

Live Demo: Từ Code Push đến Production

01 git push → GitHub Actions trigger

Push commit lên main. GitHub Actions workflow kích hoạt tự động. Quan sát Jobs chạy trong real-time.

02 DVC pull + dvc repro

Pipeline pull đúng data version. dvc repro chạy lại stages bị stale. Smart cache bỏ qua stages unchanged.

03 Train + log MLflow

Train model, log params/metrics/artifacts vào MLflow. Register vào Model Registry ở stage Staging.

04 Eval Gate: compare vs baseline

compare_models.py lấy Production baseline từ Registry. So sánh accuracy. Pass → tiếp tục. Fail → block deploy.

05 Canary deploy: 5% → 100%

Canary 5% traffic. Monitor 2 phút. Nếu P99 latency và accuracy OK → rollout 100%.

BONUS Simulate model regression → block

Cố tình degrade model. Quan sát eval gate tự động block. Zero regression lọt production.

Key Takeaways

Những ý chính cần nhớ sau buổi học hôm nay

1

MLflow + DVC = Full Reproducibility

Bất kỳ ai clone repo + dvc pull đều reproduce y hệt kết quả. MLflow track experiments, DVC track data — hai công cụ bổ sung nhau, không thay thế nhau.

2

Eval Gate là Safety Net Quan Trọng Nhất

Never deploy without comparing to baseline. Eval gate là lớp bảo vệ cuối cùng. Case study: 0 regression lỗi production trong 6 tháng sau khi implement CI/CD.

3

Canary Deployment Giảm Risk 90%

Big-bang release là nguồn gốc của sự cố. Invest vào gradual rollout: 5% → 25% → 50% → 100% với health checks. Rollback ngay khi metrics degrade.

Preview Ngày 22 & Bài Tập

Ngày 22: LLMOps & Prompt Versioning

"LangSmith, Weights & Biases Weave cho LLM-specific operations — prompt là code, phải version control"

- LangSmith: tracing, prompt hub, evaluations
- W&B Weave: LLM-specific experiment tracking
- Prompt versioning workflow trong CI/CD
- LLM regression testing: eval suite tự động

Lab #21 — Bài Tập (2 giờ)

- Setup MLflow tracking server local (SQLite backend)
- Convert training script để log params, metrics, artifacts
- Setup DVC với S3/GCS remote + pipeline 3 stages
- Viết GitHub Actions workflow: test → train → eval → deploy
- Đăng ký tài khoản LangSmith (free tier)
- Đọc trước: LangSmith docs (tracing & Prompt Hub)

Slide & template → lms.vinuni.edu.vn | AICB-P2T2 Ngày 21 | CI/CD for AI Systems